

Question No:4

Problem Description

You are given a weighted graph with **N** vertices and **M** edges. Find the total weight of its **maximum** spanning tree.

Constraints

- $1 \leq T \leq 20$
- $1 \leq N \leq 5000$
- $1 \leq M \leq 100\,000$
- $1 \leq c \leq 10\,000$

Input

The first line contains one integer **T** denoting the number of test cases. Each test case starts with a line containing **2** space-separated integers: **N** and **M**. Each of the following **M** lines contain description of one edge: three different space-separated integers: **a**, **b** and **c**. **a** and **b** are different and from 1 to **N** each and denote numbers of vertices that are connected by this edge. **c** denotes weight of this edge.

Output

For each test case output one integer - the total weight of its **maximum** spanning tree.

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
1	1
3 3	3 3
1 2 2	1 2 3
2 3 3	2 3 4
1 3 4	1 3 5
EXPECTED OUTPUT	EXPECTED OUTPUT
7	9

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  typedef struct
4  {
5      int u, v, weight;
6  } Edge;
7
8  int parent[5005];
9  int rank[5005];
10
11 int find(int x)
12 {
13     if(parent[x] != x)
14         parent[x] = find(parent[x]);
15
16     return parent[x];
17 }
18
19 void unionSet(int a, int b)
20 {
21     a = find(a);
22     b = find(b);
23
24     if(a == b)
25         return;
26
27     if(rank[a] < rank[b])
28         parent[a] = b;
```

```

29     else if(rank[a] > rank[b])
30     |     parent[b] = a;
31     else
32     {
33         parent[b] = a;
34         rank[a]++;
35     }
36 }
37
38 int compare(const void *a, const void *b)
39 {
40     Edge *e1 = (Edge *)a;
41     Edge *e2 = (Edge *)b;
42
43     return e2->weight - e1->weight;    // Descending order
44 }
45
46 int main()
47 {
48     int T;
49     scanf("%d", &T);
50
51     while(T--)
52     {
53         int N, M;
54         scanf("%d %d", &N, &M);
55

```

```

55
56     Edge *edges = malloc(M * sizeof(Edge));
57
58     for(int i = 0; i < M; i++)
59     {
60         scanf("%d %d %d",
61             &edges[i].u,
62             &edges[i].v,
63             &edges[i].weight);
64     }
65
66     // Initialize DSU
67     for(int i = 1; i <= N; i++)
68     {
69         parent[i] = i;
70         rank[i] = 0;
71     }
72
73     // Sort edges from largest weight to smallest
74     qsort(edges, M, sizeof(Edge), compare);
75
76     long long totalWeight = 0;
77     int edgesUsed = 0;
78
79     // Kruskal's algorithm
80     for(int i = 0; i < M && edgesUsed < N - 1; i++)
81     {
82         int u = edges[i].u;
83         int v = edges[i].v;
84
85         if(find(u) != find(v))
86         {
87             unionSet(u, v);
88
89             totalWeight += edges[i].weight;
90             edgesUsed++;
91         }
92     }
93
94     printf("%lld\n", totalWeight);
95
96     free(edges);
97 }
98
99     return 0;
100 }

```

Output:

Test Case-1:

	1		
	3	3	
	1	2	2
	2	3	3
	1	3	4
	7		

Test Case-2:

	1		
	3	3	
	1	2	3
	2	3	4
	1	3	5
	9		

Keshav Kumar Rauniyar
CH.SC.U4CSE25236